

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра системного проектування

Допустити до захисту
Завідувач кафедри
доц. Шувар Р. Я. _____
(підпис)
«...» 2026 р.

Кваліфікаційна робота
Бакалавр

Розробка моделі планування харчування та моніторингу споживання калорій для AI-асистента

Виконав:
студент групи ФЕІ-42
спеціальності
122 Комп'ютерні науки
Вибираний Владислав

Науковий керівник:
доц. Стахіра Р. Й. _____
(підпис)
«...» 2026 р.

Рецензент:

(підпис)

Львів 2026

АНОТАЦІЯ

У кваліфікаційній роботі розглянуто розробку веб-застосунку NutriAI, орієнтованого на планування харчування, моніторинг споживання калорій та підтримку користувача за допомогою AI-асистента. Робота поєднує аналіз предметної області персоналізованого харчування, формалізацію доменної моделі, опис архітектури сучасного full-stack застосунку та реалізацію інтелектуального модуля, який виконує пошук продуктів, аналіз статистики користувача та підтримує сценарії логуювання харчування через виклики інструментів.

У теоретичній частині досліджено сучасні підходи до побудови AI-агентів, зокрема tool-calling, guardrails, керування контекстом, ризики prompt injection та особливості використання великих мовних моделей у прикладних інформаційних системах. Практична частина містить опис стеку Next.js, Bun, Prisma, PostgreSQL, Better Auth, Elysia та React Query, а також механізмів реалізації профілю користувача, журналу харчування, планів харчування, цілей та AI-чату.

Окрему увагу приділено відтворюваності демонстраційних матеріалів: у репозиторії підготовлено сценарій формування демо-даних та автоматизованого знімання скріншотів проєкту для подальшого оновлення пояснювальної записки. Запропоновано каркас для майбутнього кількісного порівняння різних методів побудови AI-агентів у контексті nutrition assistant.

ABSTRACT

The bachelor's thesis presents the development of NutriAI, a web application for meal planning, calorie intake monitoring, and user support with an AI assistant. The work combines domain analysis of personalized nutrition, formalization of the data model, description of a modern full-stack architecture, and implementation of an intelligent assistant that can search foods, inspect nutrition statistics, and support food logging workflows through tool calls.

The theoretical part focuses on modern approaches to AI agents, including tool calling, context management, guardrails, prompt injection risks, and the use of large language models in applied software systems. The practical part describes the selected technology stack based on Next.js, Bun, Prisma, PostgreSQL, Better Auth, Elysia, and React Query, as well as the implementation of user profile management, nutrition logs, meal plans, goals, and AI chat.

Special attention is paid to reproducible documentation assets: the repository includes a demo-data generator and an automated screenshot pipeline so the report can be updated with fresh interface illustrations. A scaffold for future quantitative comparison of agent-building approaches is also prepared.

ЗМІСТ

1. Вступ	6
2. Аналіз стану проблемної області	8
2.1. Системи моніторингу харчування та калорій	8
2.2. Підходи до meal planning	8
2.3. AI-асистенти в прикладних інформаційних системах	9
2.4. Критичний аналіз сучасних підходів до побудови агентів	9
2.5. Висновки до розділу	9
3. Теоретична частина: інформаційне та математичне забезпечення	11
3.1. Модель предметної області	11
3.2. Структура даних та датамодель	12
3.3. Модель розрахунку калорійності та макронутрієнтів	13
3.4. Методи побудови статистики та адгезії до цілей	13
3.5. Теоретичні засади побудови AI-агентів	14
3.6. Контекст, пам'ять, guardrails та безпека	15
3.7. Особливості створення агентів у веб-застосунках	15
4. Програмне забезпечення та архітектура системи	16
4.1. Обґрунтування вибору стеку	16
4.2. Загальна архітектура	16
4.3. Модулі користувацького інтерфейсу	17
4.4. AI-модуль та маршрут server/routes/chat.ts	17
4.5. Схема обробки AI-запиту	18
5. Програмна реалізація та аналіз отриманих результатів	19
5.1. Реалізовані користувацькі сценарії	19
5.2. Демонстрація інтерфейсу застосунку	19
5.3. Сценарії використання AI-асистента	26
5.4. Переваги та обмеження реалізованої системи	27
5.5. Технічні ризики	27
5.6. Каркас для порівняння різних методів побудови агентів	27
5.7. Методика майбутнього експериментального порівняння	28
5.8. Висновки до розділу	28
6. Висновки	29
7. Список використаних джерел	30
8. Додаток А. Опис репозиторію та команд відтворення	31
9. Додаток Б. Примітки щодо подальшого розвитку	32

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ, СИМВОЛІВ, СКОРОЧЕНЬ І ТЕРМІНІВ

AI (Artificial Intelligence) — штучний інтелект; сукупність методів, моделей і програмних засобів, що забезпечують імітацію окремих інтелектуальних функцій людини, зокрема аналіз вхідних даних, побудову висновків, генерацію відповідей і підтримку прийняття рішень у межах інформаційної системи.

API (Application Programming Interface) — прикладний програмний інтерфейс; формалізований набір правил, структур даних і викликів, за допомогою яких окремі програмні модулі або зовнішні системи взаємодіють між собою та обмінюються даними.

BMR (Basal Metabolic Rate) — базальний обмін речовин; мінімальний рівень добових енерговитрат організму, необхідний для підтримання життєво важливих функцій у стані фізіологічного спокою.

BMI (Body Mass Index) — індекс маси тіла; розрахунковий антропометричний показник, що визначається як відношення маси тіла до квадрату зросту та використовується для попередньої оцінки відповідності маси тіла зросту людини.

CRUD (Create, Read, Update, Delete) — базові операції роботи з даними, що охоплюють створення нових записів, читання наявної інформації, оновлення збережених значень і видалення об'єктів із системи.

ER-діаграма (Entity-Relationship diagram) — діаграма «сутність-зв'язок»; графічна модель предметної області, яка відображає основні сутності системи, їх атрибути та логічні зв'язки між ними.

LLM (Large Language Model) — велика мовна модель; різновид моделі штучного інтелекту, навчений на значних обсягах текстових даних і призначений для розуміння природної мови, генерації тексту, узагальнення інформації та підтримки діалогової взаємодії.

MCP (Model Context Protocol) — протокол передавання контексту моделі; підхід до стандартизованої взаємодії мовної моделі із зовнішніми інструментами, джерелами даних і службами, що дає змогу керовано передавати контекст і отримувати результати виконання дій.

ORM (Object-Relational Mapping) — об'єктно-реляційне відображення; підхід до роботи з базою даних, за якого записи реляційних таблиць подаються у вигляді програмних об'єктів, а операції над даними виконуються через абстракції прикладного коду.

TDEE (Total Daily Energy Expenditure) — загальні добові енерговитрати; сумарна кількість енергії, яку організм витрачає протягом доби з урахуванням базального обміну речовин, рівня фізичної активності та інших енергетичних витрат.

1. ВСТУП

Розвиток цифрових сервісів у сфері здоров'я та self-tracking суттєво змінив спосіб, у який користувачі планують харчування, контролюють споживання калорій та оцінюють власні звички. Сучасні мобільні й веб-інструменти дають змогу фіксувати прийоми їжі, розраховувати харчову цінність продуктів, будувати плани харчування та відстежувати виконання цілей. Проте в більшості випадків ці системи або залишаються простими журналами споживання, або вимагають від користувача значного ручного введення даних і самостійної інтерпретації результатів. З погляду охорони здоров'я контроль якості харчування та енергетичного балансу залишається критично важливим для профілактики надмірної маси тіла та формування здорової поведінки [1], [2].

Одночасно із розвитком систем моніторингу харчування відбувається стрімке поширення великих мовних моделей і прикладних AI-асистентів. На відміну від звичайного чат-інтерфейсу, сучасний AI-асистент може не лише генерувати текстові поради, а й працювати з даними застосунку, аналізувати поточний стан користувача, виконувати виклики інструментів, ініціювати обчислення та підтримувати сценарії прийняття рішень. У контексті систем планування харчування це означає можливість перейти від пасивної візуалізації показників до активної допомоги: аналізу дефіциту білка, пошуку продуктів, формування підказок щодо meal planning і пояснення прогресу.

Актуальність роботи визначається потребою у програмних системах, що поєднують класичний облік харчування з інтелектуальним супроводом користувача. Для спеціальності «Інженерія програмного забезпечення» тема є релевантною також через необхідність комплексно поєднати доменне моделювання, веб-архітектуру, роботу з базою даних, інтерфейсну взаємодію та інтеграцію AI-компонентів.

Мета роботи полягає у розробці моделі планування харчування та моніторингу споживання калорій для AI-асистента, а також у реалізації програмного застосунку, який підтримує цю модель і надає користувачеві інструменти для ведення харчового журналу, планування раціону й аналізу прогресу.

Для досягнення поставленої мети сформульовано такі **завдання роботи**:

1. провести аналіз предметної області систем моніторингу харчування та сучасних AI-асистентів;
2. описати модель даних для персоналізованого харчування, журналу споживання, цілей і планів;

3. обґрунтувати вибір стеку та архітектури веб-застосунку;
4. реалізувати модулі профілю, продуктового каталогу, food log, meal plans, goals та аналітики;
5. реалізувати AI-модуль з доступом до прикладних інструментів;
6. підготувати автоматизований сценарій формування демонстраційних матеріалів для пояснювальної записки.

Об'єкт дослідження — процес цифрової підтримки користувача під час планування харчування і контролю споживання калорій.

Предмет дослідження — моделі даних, алгоритми обчислення харчових показників, архітектурні рішення та методи інтеграції AI-асистента в систему планування харчування.

Методи дослідження включають аналіз предметної області, порівняльний аналіз програмних рішень, моделювання структури даних, проєктування full-stack архітектури, реалізацію прикладних алгоритмів розрахунку макронутрієнтів і статистики, а також методи побудови AI-агентів на основі tool calling та контекстних інструкцій.

Елементи новизни полягають у поєднанні в одному застосунку персоналізованого планування харчування, аналітики харчових показників та AI-модуля, що працює не із зовнішнім абстрактним знанням, а з реальним станом користувача в межах доменної моделі проєкту.

Галузь застосування результатів — системи self-tracking, nutrition assistant, wellness-платформи, персональні дашборди здоров'я та навчальні проєкти, що поєднують веб-розробку з AI-компонентами.

Прогнозні напрями розвитку включають кількісне порівняння різних агентних підходів, інтеграцію ширшого харчового каталогу, рекомендаційних моделей і розширення системи безпеки AI-викликів.

2. АНАЛІЗ СТАНУ ПРОБЛЕМНОЇ ОБЛАСТІ

2.1. Системи моніторингу харчування та калорій

Системи обліку харчування зазвичай будуються навколо кількох базових доменних сутностей: користувач, профіль, каталог продуктів, журнал прийомів їжі, цільові показники та історія змін. Найпоширеніший сценарій використання полягає в тому, що користувач вводить продукти, вибирає розмір порції, після чого система агрегує калорійність і макронутрієнти за день. Практична цінність такого підходу очевидна: користувач бачить реальне відхилення від плану, може контролювати білок, жири та вуглеводи, а також зіставляти раціон із власними цілями.

Втім, класичні *calorie trackers* мають обмеження. По-перше, якість користувацького досвіду сильно залежить від повноти каталогу продуктів і зручності повторного використання вже введених записів. По-друге, сам факт накопичення цифр ще не означає, що користувач отримує дієві підказки: часто інтерфейс лише демонструє факт недобору білка чи перевищення калорійності, але не пояснює, як це виправити. По-третє, більшість систем розглядає планування раціону і логування їжі як два слабо пов'язані модулі.

Для NutriAI важливим стало поєднання цих аспектів у єдиний робочий цикл: створення або вибір продуктів, планування *meal plan*, перенесення плану в журнал фактичного споживання і подальший аналіз результатів. Така цілісність зменшує кількість ручних дій та підвищує прикладну цінність застосування.

2.2. Підходи до *meal planning*

Meal planning у програмних системах може реалізовуватися на різних рівнях складності. Найпростіший рівень — це статичний перелік страв або продуктів на конкретний день. Наступний рівень — тижневий план з повторним використанням блоків, наприклад сніданків і вечерь. Найбільш складні підходи доповнюють цей процес оптимізацією за калорійністю, макронутрієнтами, бюджетом або доступністю продуктів.

У межах поточного проєкту обрано прагматичну модель: *meal plan* описується як набір елементів для конкретного дня тижня з указанням *meal type*, порції та примітки. Це рішення не перевантажує користувача повноцінною системою раціонального оптимізатора, але водночас зберігає повторюваність і дає можливість легко переносити план у *food log*. Такий підхід добре відповідає навчальному *full-stack* проєкту, оскільки

дозволяє реалізувати достатньо багату доменну модель без надмірного ускладнення математичної частини.

2.3. AI-асистенти в прикладних інформаційних системах

Інтеграція AI-асистента в бізнес- або персональний застосунок є якісно іншою задачею, ніж створення звичайного чат-бота. Якщо загальний чат-бот працює лише на текстовому контексті, то прикладний асистент має доступ до внутрішніх даних системи, а отже може виконувати конкретні дії: шукати продукти, аналізувати статистику, логувати прийоми їжі, формувати рекомендації та пояснювати користувачу причини відхилень.

Така інтеграція потребує чіткого визначення інструментів, якими AI має право користуватися. У NutriAI асистент працює через набір конкретних функцій: пошук продуктів у каталозі, читання денного журналу, аналіз статистики за період, обчислення рекомендованих макросів, отримання профілю та активних цілей. Це дозволяє поєднати гнучкість LLM з контрольованим доступом до даних.

2.4. Критичний аналіз сучасних підходів до побудови агентів

У сучасній практиці можна виділити кілька типових підходів до побудови AI-агентів. Їх розвиток підштовхується появою спеціалізованих фреймворків і протоколів на кшталт MCP, LangGraph, AutoGen та Agents SDK [3], [4], [5], [6]:

1. **single-agent** — одна модель отримує контекст і самостійно виконує послідовність викликів;
2. **tool-calling agent** — LLM використовує явний реєстр інструментів і викликає їх структуровано;
3. **workflow agent** — система керується заздалегідь заданим графом або пайплайном кроків;
4. **multi-agent** — кілька агентів отримують окремі ролі й координуються між собою.

Single-agent підхід є найдешевшим у реалізації, але найслабше контролюється. Multi-agent архітектури добре підходять для складних задач з явною спеціалізацією ролей, але для applied nutrition assistant часто є надлишковими через вартість, затримки й потребу складної оркестрації. Workflow-підходи виграють у передбачуваності, проте поступаються гнучкістю для розмовних сценаріїв. У межах NutriAI найбільш виправданим став tool-calling підхід: він забезпечує баланс між природною мовою, доступом до доменних інструментів і контрольованістю дій.

2.5. Висновки до розділу

Аналіз предметної області показує, що прикладна цінність nutrition assistant зростає тоді, коли система не просто відображає статистику, а об'єднує планування, логування, аналітику й AI-підтримку в єдиний цикл. Саме цю нішу займає проєкт NutriAI: він поєднує типові функції nutrition tracker з інструментальним AI-асистентом, який працює безпосередньо з даними користувача.

3. ТЕОРЕТИЧНА ЧАСТИНА: ІНФОРМАЦІЙНЕ ТА МАТЕМАТИЧНЕ ЗАБЕЗПЕЧЕННЯ

3.1. Модель предметної області

У центрі предметної області знаходиться користувач, для якого задаються персональні параметри: стать, зріст, вага, рівень фізичної активності та цільові показники за калоріями і макронутрієнтами. Профіль користувача визначає контекст персоналізації для всіх інших модулів.

Наступним блоком є **каталог продуктів**, який містить харчові характеристики елементів раціону: порцію, одиницю виміру, калорійність, білки, жири, вуглеводи, а також додаткові показники на кшталт клітковини, цукру чи натрію. На основі каталогу будуються два основні процеси: фактичне логування споживання та планування майбутніх прийомів їжі.

Журнал харчування моделює фактичне споживання продуктів за датою. Кожен запис журналу містить набір елементів з вказанням продукту, типу прийому їжі, кількості порцій і приміток. Завдяки цій структурі система може обчислювати сумарні денні показники та порівнювати їх із цілями користувача.

План харчування описує запланований раціон за днями тижня. Кожен елемент плану пов'язаний із конкретним продуктом, днем тижня та meal type. Це дозволяє реалізувати як повторюваний weekly meal prep, так і точкові сценарії складання раціону.

Цілі моделюють керовані орієнтири користувача: ціль по білку, калорійності, вазі чи довільному показнику. Вони потрібні не лише для відображення в UI, а й для побудови пояснень AI-асистента, який має знати, що саме вважається успішним результатом.

Чат і пов'язані з ним повідомлення забезпечують збереження історії взаємодії користувача з AI-асистентом. Важливо, що повідомлення асистента можуть містити не лише текст, а й історію tool invocations та результатів викликів.

Сутність	Призначення	Ключові поля	Основні зв'язки
User	Обліковий запис користувача	id, email, name	1:1 з UserProfile, 1:M з FoodLog, MealPlan, Goal, Conversation
UserProfile	Персоналізація цілей	height, weight, activityLevel, targetCalories	належить одному User
Food	Каталог продуктів	name, servingSize, calories, protein, carbs, fat	використовується в FoodLogItem і MealPlanItem
FoodLog	Журнал за конкретну дату	date, notes	містить FoodLogItem
MealPlan	План харчування на період	name, startDate, endDate, isActive	містить MealPlanItem
Goal	Ціль користувача	type, targetValue, currentValue, status	належить User
Conversation / Message	Історія взаємодії з AI	title, role, content, toolCalls	належить User

Таблиця 1: Ключові сутності предметної області NutriAI.

3.2. Структура даних та датамодель

Фізичне зберігання доменних даних реалізовано через Prisma та PostgreSQL. Перевага такого підходу полягає у тому, що Prisma-схема одночасно виступає і декларацією структури, і джерелом типізованого клієнта для TypeScript-коду. У практичному сенсі це зменшує ризик неузгодженості між моделями фронтенду, API та бази даних.

У схемі проєкту є кілька важливих груп моделей:

1. auth-моделі Better Auth: User, Session, Account, Verification;
2. доменні моделі профілю й налаштувань: UserProfile, UserSettings;
3. моделі харчування: Food, FoodLog, FoodLogItem, MealPlan, MealPlanItem, UserFoodFavorite;
4. аналітичні моделі взаємодії та цілей: Goal, Conversation, Message.

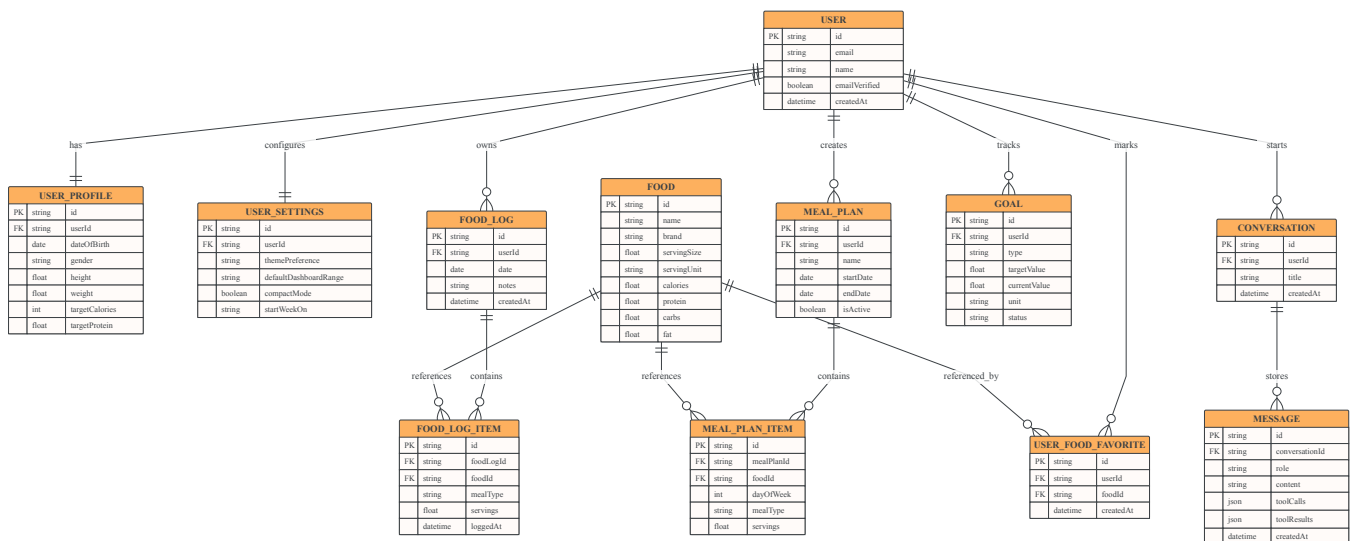


Рисунок 1: ER-діаграма основних сутностей NutriAI на основі Prisma-схеми.

3.3. Модель розрахунку калорійності та макронутрієнтів

Базовою математичною операцією системи є агрегування харчових показників за набором записів food log. Якщо для продукту відомі калорійність і макронутрієнти на одну умовну порцію, то для фактичного запису з кількістю servings підсумкові значення розраховуються множенням показників на кількість порцій:

$$C_{\text{item}} = C_{\text{food}} \cdot s$$

$$P_{\text{item}} = P_{\text{food}} \cdot s$$

$$H_{\text{item}} = H_{\text{food}} \cdot s$$

$$F_{\text{item}} = F_{\text{food}} \cdot s$$

Денні підсумки визначаються сумою внесків усіх елементів журналу за обрану дату:

$$C_{\text{day}} = \sum_{i=1}^n C_i$$

$$P_{\text{day}} = \sum_{i=1}^n P_i$$

Реалізаційно ці обчислення інкапсульовано в бібліотечному модулі lib/nutrition-analytics.ts, що містить функції sumNutritionTotals, buildDailyTotals, buildRangeStats, buildPeriodSummary та похідні допоміжні обчислення.

3.4. Методи побудови статистики та адгезії до цілей

Окрім денних сум, NutriAI реалізує серію похідних статистик, необхідних для інтерпретації поведінки користувача:

1. середні показники за період;
2. кількість днів з логуванням;
3. поточний і найдовший streak;
4. adherence до цільових значень за калоріями та макронутрієнтами;
5. weekday averages для пошуку шаблонів поведінки;
6. meal type breakdown для аналізу розподілу калорійності між сніданком, обідом, вечерею і перекусами;
7. best day / lowest day для виділення екстремальних випадків.

Таке представлення статистики є принципово важливим для AI-асистента. Якщо система зберігає лише «голі» денні суми, LLM доведеться щоразу самостійно інтерпретувати числовий масив. Якщо ж backend одразу надає підготовлені агрегати, асистент може будувати більш надійні пояснення й рекомендації.

3.5. Теоретичні засади побудови AI-агентів

Поняття **agentic AI** у прикладному програмному забезпеченні означає модель поведінки, за якої LLM не лише відповідає текстом, а виконує послідовність кроків для досягнення цілі користувача. Зазвичай цей цикл включає:

1. сприйняття запиту та поточного контексту;
2. планування або декомпозицію задачі;
3. вибір інструменту;
4. виклик інструменту та аналіз результату;
5. формування відповіді або наступного кроку.

У найпростішому варіанті агент взаємодіє з програмою через **tool use** або **function calling**. Розробник описує доступні інструменти у вигляді структурованих функцій з параметрами та обмеженнями, а модель вирішує, коли і з якими аргументами їх викликати. Перевага такого підходу полягає в тому, що всі реальні дії агента проходять через контрольовані точки інтеграції. У літературі з AI engineering саме така комбінація структурованих інструментів, контексту та перевірюваних кроків розглядається як практична основа production-oriented AI applications [7].

Для NutriAI це означає, що LLM не має прямого доступу до бази даних. Натомість вона користується набором доменних інструментів: searchFoods, logFood, getDailyLog, getUserProfile, getUserStats, getActiveGoals, calculateMacros. Кожен інструмент має чіткий контракт і повертає результат у структурованому вигляді.

3.6. Контекст, пам'ять, guardrails та безпека

Якість роботи AI-асистента залежить не лише від самої моделі, а й від того, який контекст отримує LLM. У NutriAI системна інструкція доповнюється контекстом профілю, цілей і харчового каталогу користувача. Завдяки цьому асистент не відповідає абстрактно, а орієнтується на реальні параметри конкретного акаунта.

Пам'ять у такій системі реалізується на двох рівнях. Перший рівень — історія повідомлень у межах conversation. Другий — доменні дані користувача, які зберігаються в базі і можуть бути повторно витягнуті через інструменти. Саме поєднання розмовної та доменної пам'яті робить applied assistant корисним.

Guardrails — це правила та механізми, що обмежують поведінку агента. У контексті NutriAI guardrails реалізуються через:

1. явний перелік дозволених інструментів;
2. типізацію аргументів інструментів;
3. відсутність прямого SQL-доступу;
4. автентифікаційний контекст користувача;
5. обмеження кількості tool roundtrips.

Окрему проблему становить **prompt injection**: модель може отримати шкідливу інструкцію з контексту або зовнішнього джерела. У прикладних системах це небезпечно тим, що асистент потенційно має доступ до дій у системі. Тому для production-oriented агентів недостатньо лише довіряти LLM; потрібно мінімізувати surface area інструментів, журналювати дії, відокремлювати читання від модифікації та, за потреби, додавати підтвердження на критичні операції.

3.7. Особливості створення агентів у веб-застосунках

Інтеграція AI в веб-застосунок має кілька важливих особливостей:

1. відповідь асистента повинна бути прив'язана до авторизованого користувача;
2. інструменти мусять повертати доменно значущі дані, а не надмірно сирі структури;
3. UI має відображати не лише фінальний текст, а й діяльність агента;
4. необхідно балансувати між latency та корисністю відповіді.

У NutriAI це реалізовано через окремий маршрут `server/routes/chat.ts`, збереження conversation history, рендеринг tool activity у UI та інструментальний контракт на стороні сервера.

4. ПРОГРАМНЕ ЗАБЕЗПЕЧЕННЯ ТА АРХІТЕКТУРА СИСТЕМИ

4.1. Обґрунтування вибору стеку

Для реалізації NutriAI обрано стек, який поєднує сучасний UI-рівень, типізований сервер, зручну роботу з даними та швидкий цикл розробки [8], [9], [10], [11], [12], [13]:

1. **Next.js 16** використовується як основа веб-застосунку з App Router;
2. **React 19** є базовою бібліотекою побудови інтерфейсу;
3. **Bun** застосовується для запуску скриптів, тестів та керування залежностями;
4. **Prisma** забезпечує роботу з PostgreSQL через типізований ORM;
5. **Better Auth** відповідає за email/password автентифікацію;
6. **Elysia** використовується для побудови API під префіксом /api;
7. **React Query** організовує клієнтський доступ до даних, інвалідує кеш і синхронізує стан між сторінками.

Такий набір інструментів добре відповідає природі проєкту: застосунок має виражену CRUD-частину, потребує швидкого оновлення статистики в UI, складних форм, сесій користувача та AI-маршруту зі stateful conversation.

4.2. Загальна архітектура

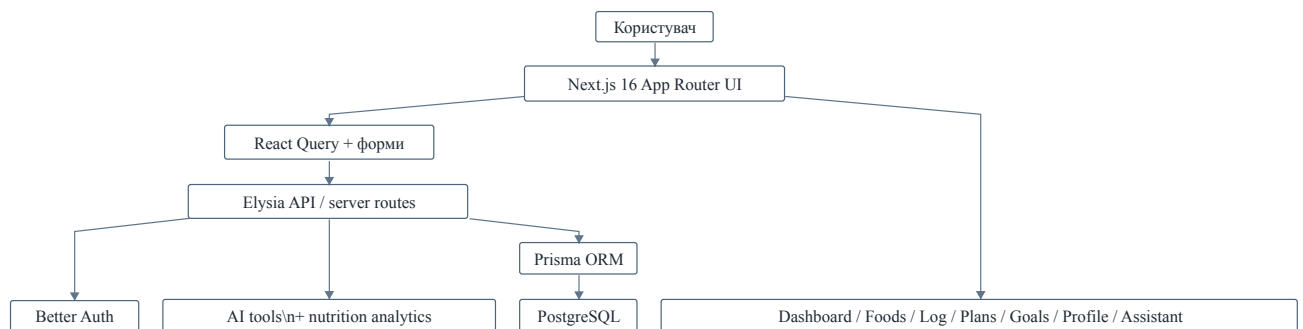


Рисунок 2: Діаграма загальної архітектури NutriAI.

Архітектурно застосунок поділяється на три рівні:

1. **presentation layer** — сторінки в каталозі app/ та UI/feature-компоненти в components/;
2. **application/API layer** — маршрути Elysia в server/routes/;
3. **data layer** — Prisma та PostgreSQL, доповнені бібліотечними модулями аналітики.

Цей поділ дозволяє зберігати чітке розмежування між відображенням, прикладною логікою і даними. Для дипломного проєкту це важливо ще й методологічно: структура

застосунку напряду відповідає вимогам до демонстрації уміння проєктувати архітектуру, модулі та життєвий цикл ПЗ. З точки зору класичної інженерії програмного забезпечення таке розділення покращує керованість змін, тестованість і прозорість відповідальностей між компонентами [14].

4.3. Модулі користувацького інтерфейсу

Функціональна поверхня NutriAI охоплює такі основні сторінки:

1. Dashboard — оперативна панель зі зведеними показниками, streak, quick actions і трендами;
2. Foods — каталог продуктів з фільтрами, порівнянням, логуванням і додаванням до плану;
3. Food Log — щоденний журнал споживання;
4. Plans — тижневі meal plans і shopping list;
5. Goals — активні й історичні цілі;
6. Profile — персональні параметри та цільові значення;
7. Settings — керування темою та поведінкою workspace;
8. Assistant — інтерфейс AI-чату.

З позиції пояснювальної записки важливо, що ці сторінки не є ізольованими. У репозиторії помітна тенденція до побудови наскрізного daily loop: profile задає персональні норми, foods формує довідник, plans допомагає наперед спланувати раціон, log фіксує факт, dashboard і progress інтерпретують результат, assistant допомагає пройти цей цикл із меншим когнітивним навантаженням.

4.4. AI-модуль та маршрут `server/routes/chat.ts`

Ключовим модулем для теми роботи є `server/routes/chat.ts`. Саме тут реалізовано AI-шар, який поєднує LLM-провайдера з доменними інструментами проєкту. Маршрут виконує кілька функцій:

1. зберігає список conversation та окремі message;
2. формує системний контекст із профілю, цілей та каталогу продуктів;
3. описує інструменти для AI;
4. запускає `streamText` із заданою моделлю;
5. зберігає tool calls і tool results разом із відповіддю асистента.

Архітектурно це вдале рішення, тому що AI-модуль не дублює бізнес-логіку в окремому сервісі, а перевикористовує ті самі Prisma-запити й аналітичні функції, що й решта застосунку.

Інструмент	Призначення	Практична користь
searchFoods	пошук продуктів за назвою або брендом	асистент може підказувати реальні записи з каталогу
logFood	створення елемента журналу	підтримка швидкого логування через natural language
getDailyLog	читання фактичного журналу за день	аналіз денного раціону
getUserProfile	отримання персональних параметрів	персоналізовані рекомендації
getUserStats	аналітика за період	аналіз прогресу, streak та adherence
getActiveGoals	отримання активних цілей	зв'язування відповіді з актуальним завданням користувача
calculateMacros	обчислення орієнтовних макросів	пояснення норм і цільових значень

Таблиця 2: Набір інструментів AI-асистента в NutriAI.

4.5. Схема обробки AI-запиту

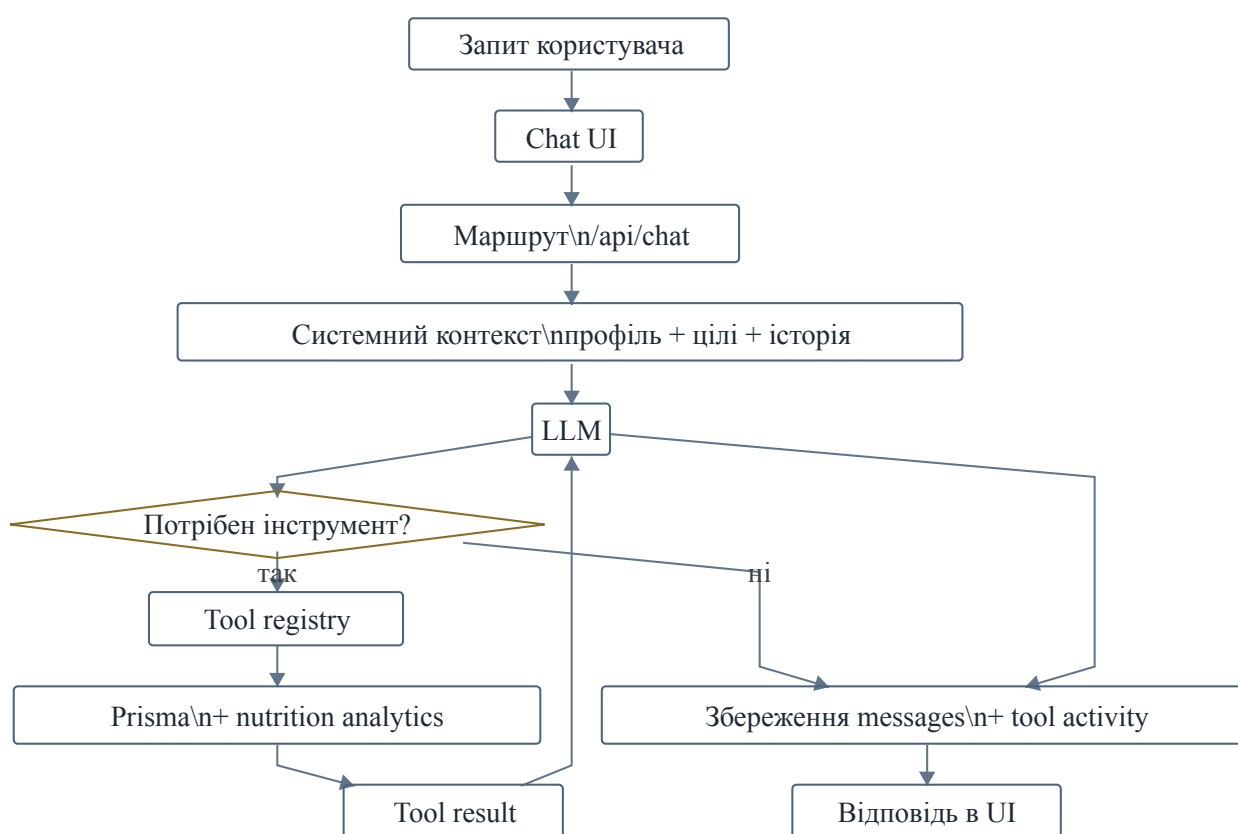


Рисунок 3: Діаграма обробки AI-запиту в NutriAI.

Ця схема показує, що LLM не взаємодіє з даними напряму. Усі дії проходять через серверний registry інструментів, а результати зберігаються в історії conversation. Це одночасно підвищує контрольованість і покращує UX, бо інтерфейс може показувати, які саме дії виконав асистент.

5. ПРОГРАМНА РЕАЛІЗАЦІЯ ТА АНАЛІЗ ОТРИМАНИХ РЕЗУЛЬТАТІВ

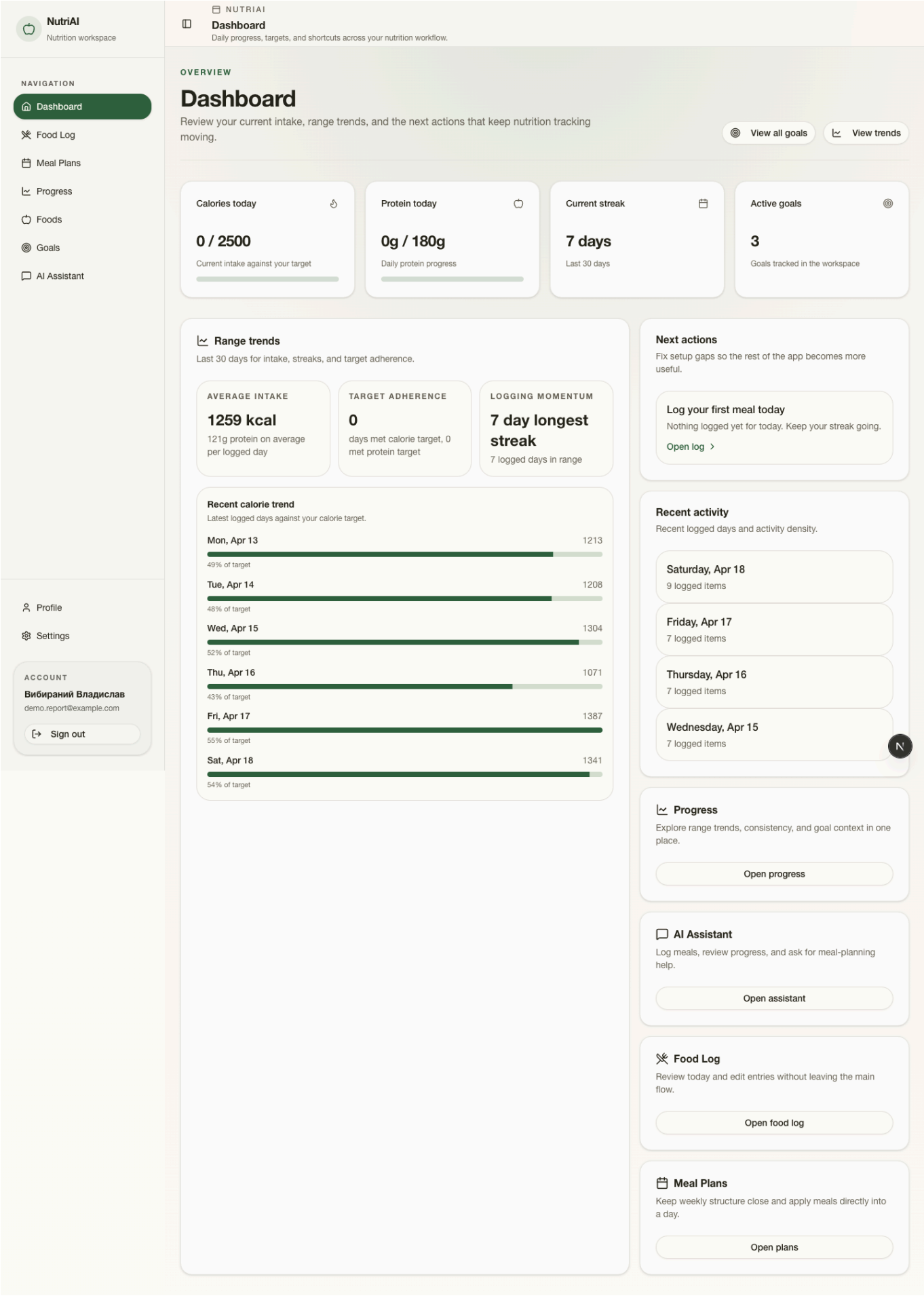
5.1. Реалізовані користувацькі сценарії

У поточному стані проєкту реалізовано повний мінімально життєздатний цикл роботи з харчуванням:

1. користувач створює або редагує профіль і встановлює цільові значення;
2. наповнює або використовує каталог продуктів;
3. формує тижневий meal plan;
4. переносить планові продукти в food log;
5. відстежує прогрес через dashboard, goals та аналітичні зведення;
6. звертається до AI-асистента по пояснення або автоматизацію окремих дій.

Сильна сторона поточної реалізації полягає в інтегрованості модулів: аналітика використовує ті самі записи журналу, що й dashboard; assistant бере контекст із профілю, цілей і каталогу; foods пов'язаний і з плануванням, і з логуванням.

5.2. Демонстрація інтерфейсу застосунку



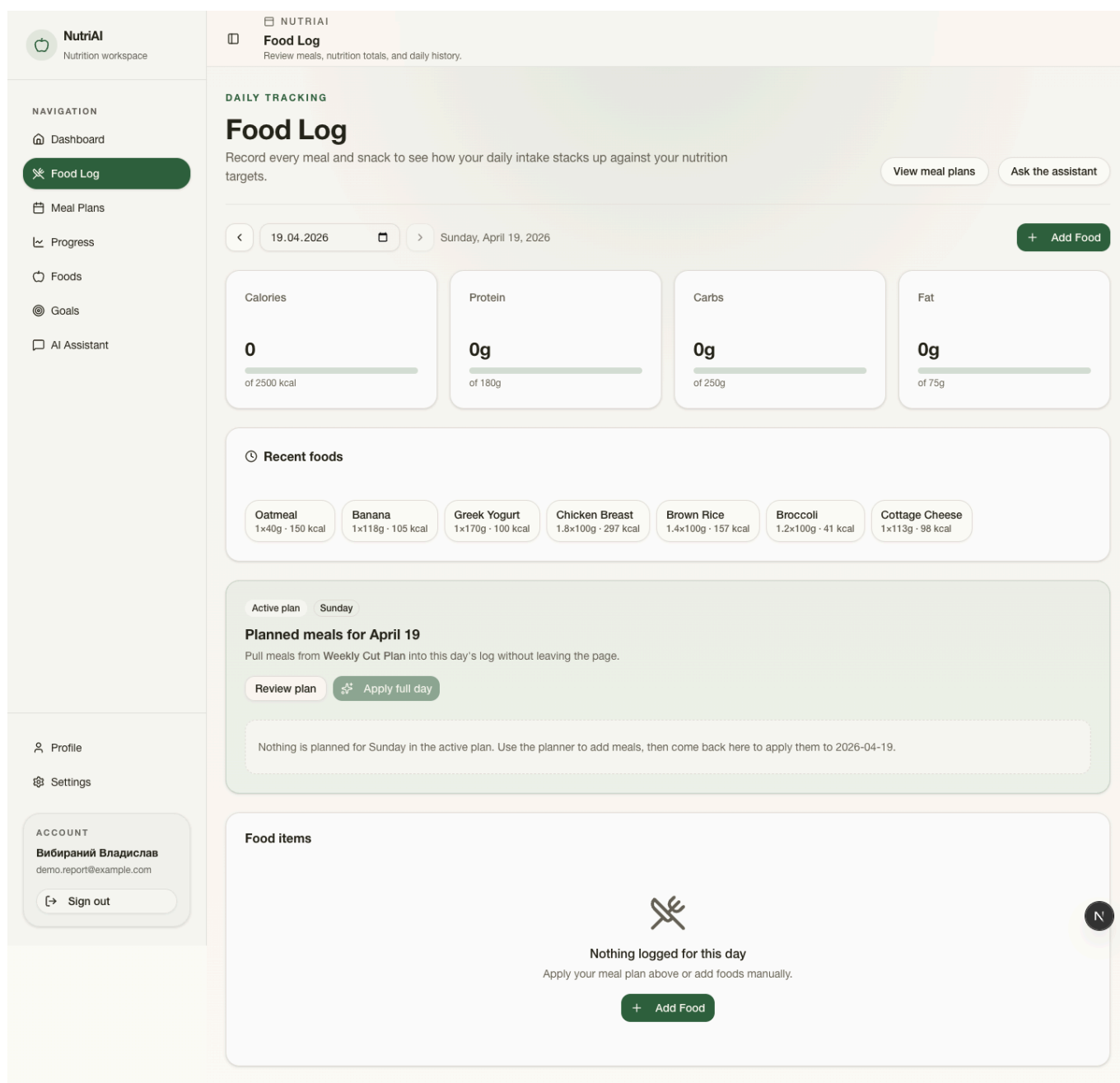


Рисунок 5: Сторінка food log для контролю фактичного споживання продуктів за день.

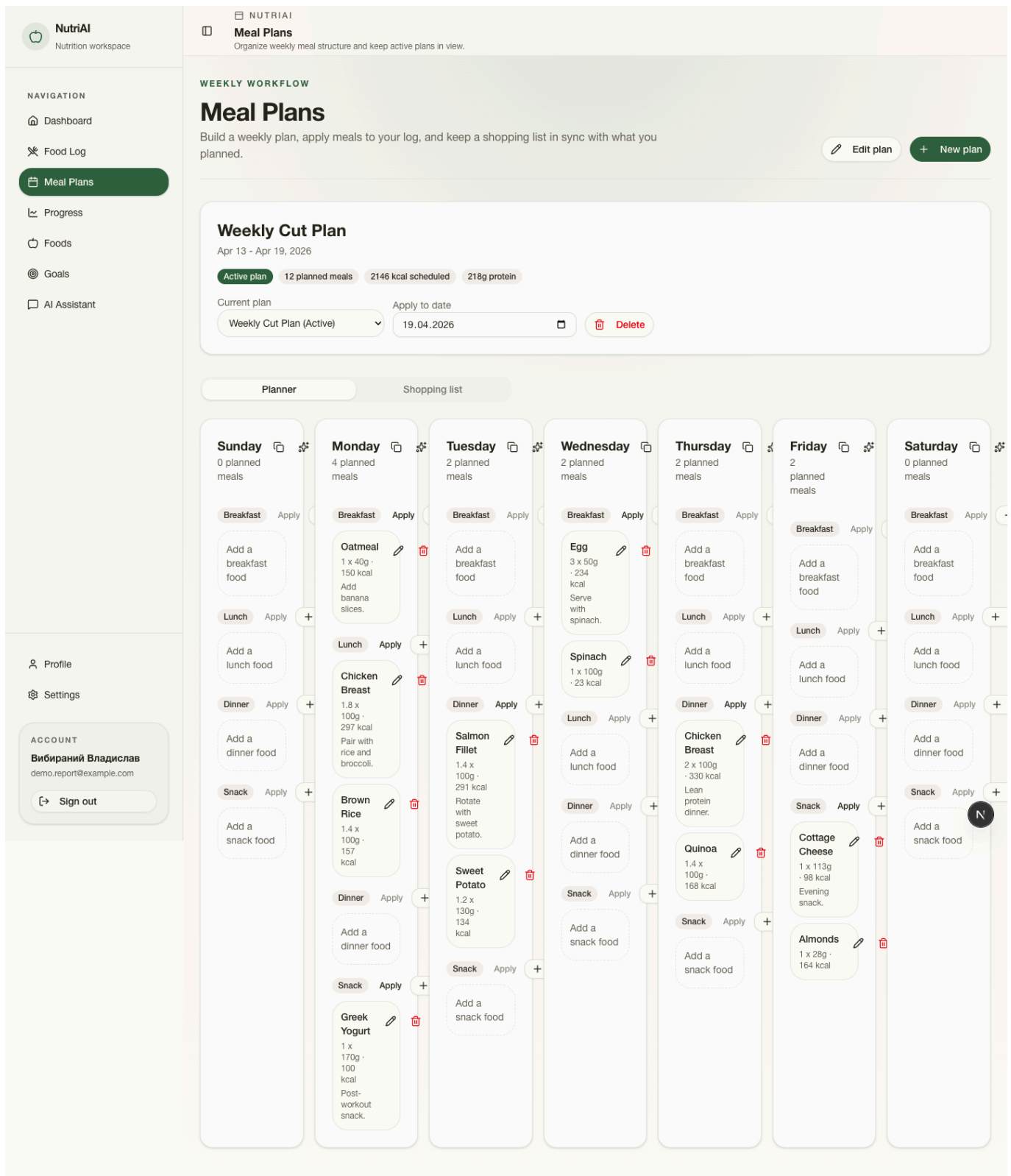


Рисунок 6: Модуль планування харчування з weekly meal plan та shopping list.

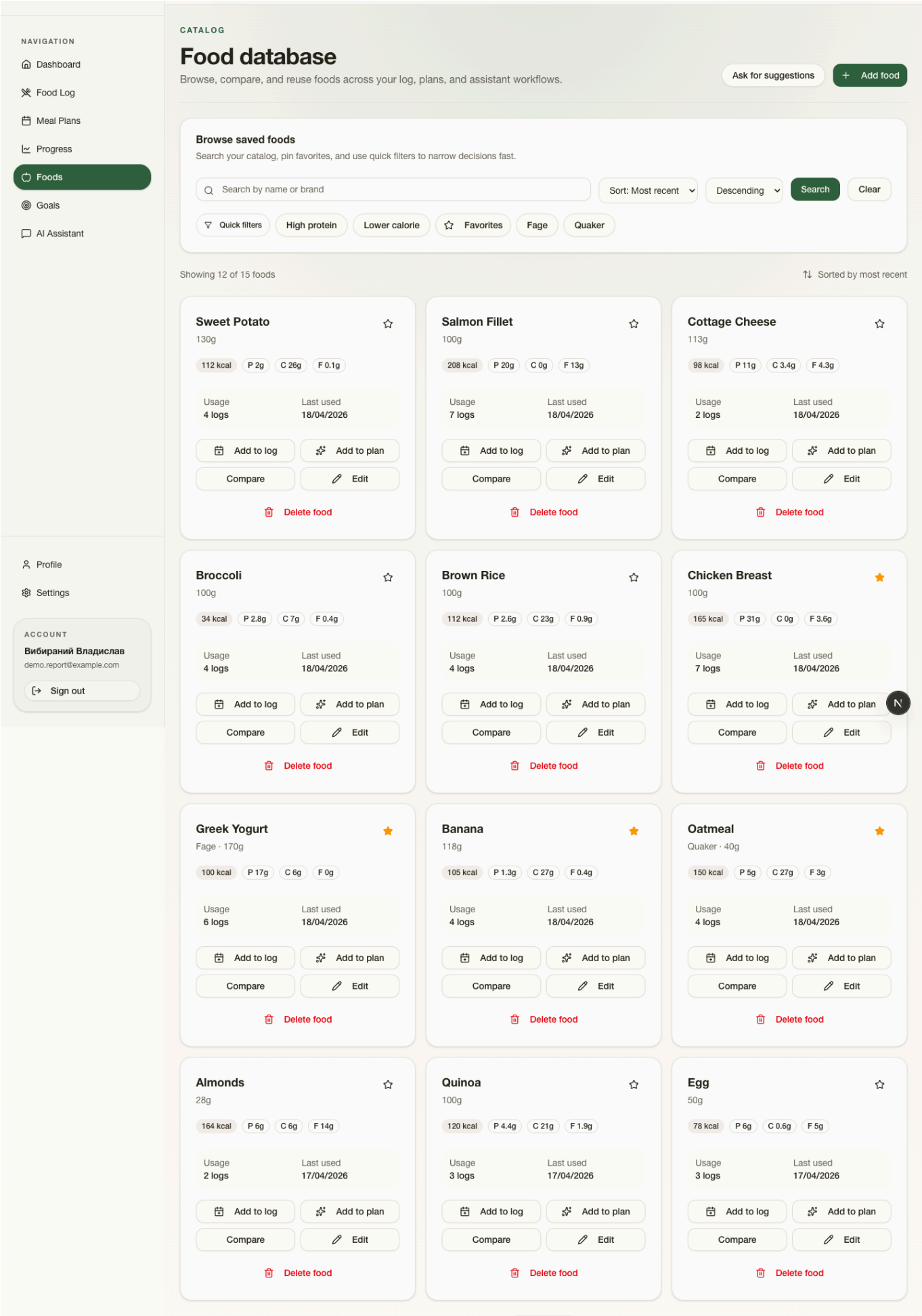


Рисунок 7: Каталог продуктів із фільтрами, фаворитами та діями для логування/планування. 23

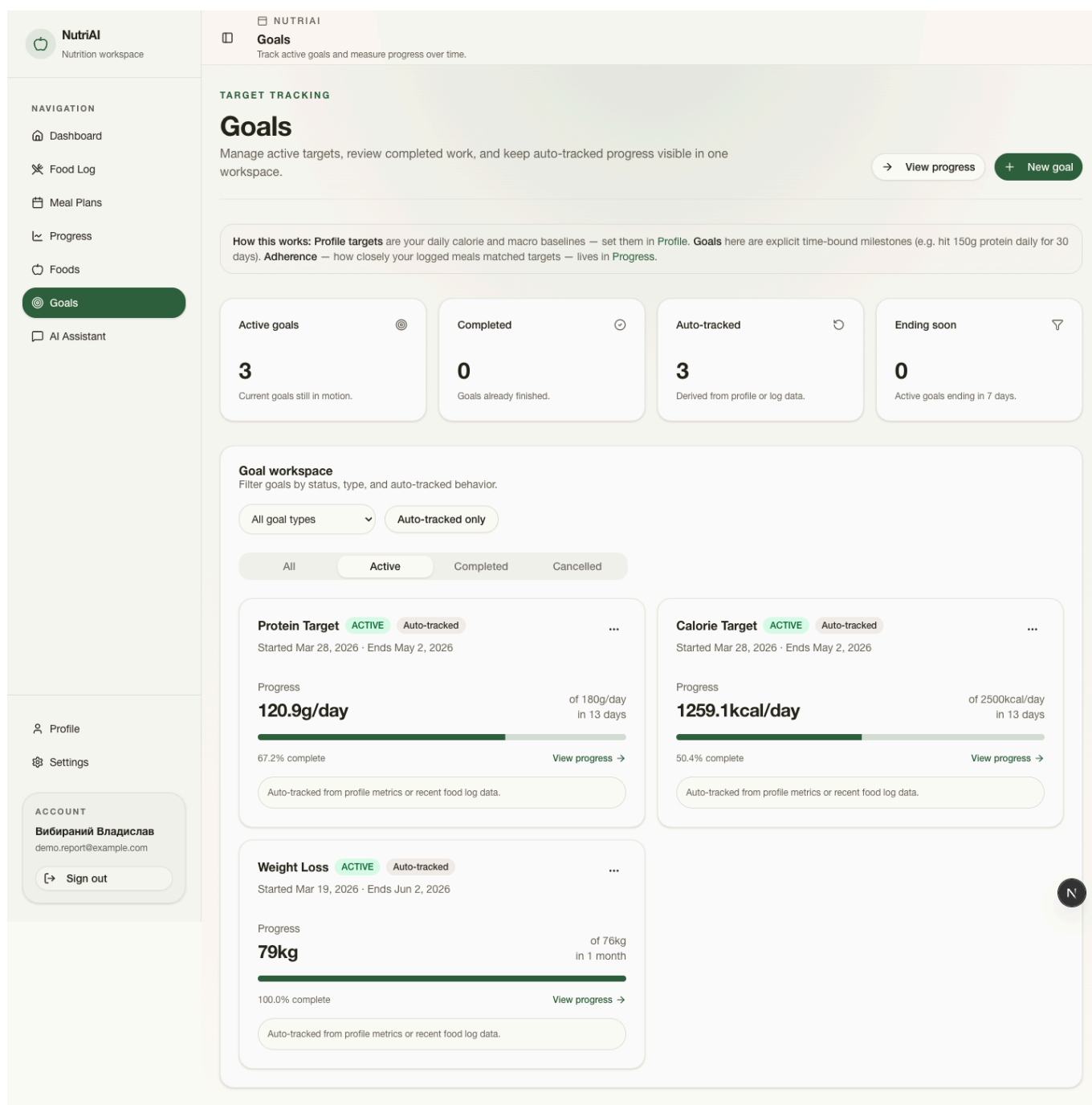


Рисунок 8: Сторінка керування цілями користувача.

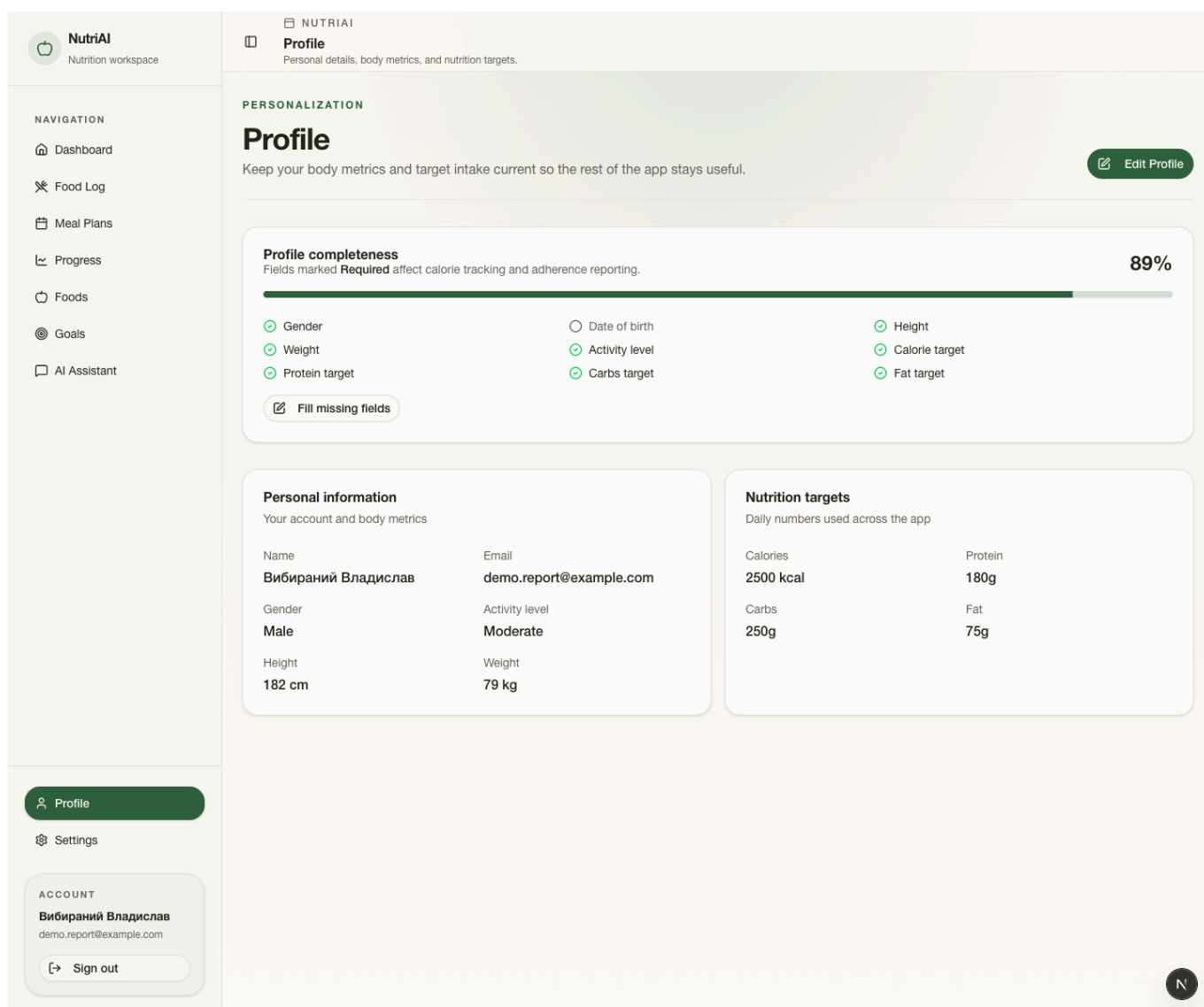


Рисунок 9: Профіль користувача з персональними параметрами та цільовими значеннями.

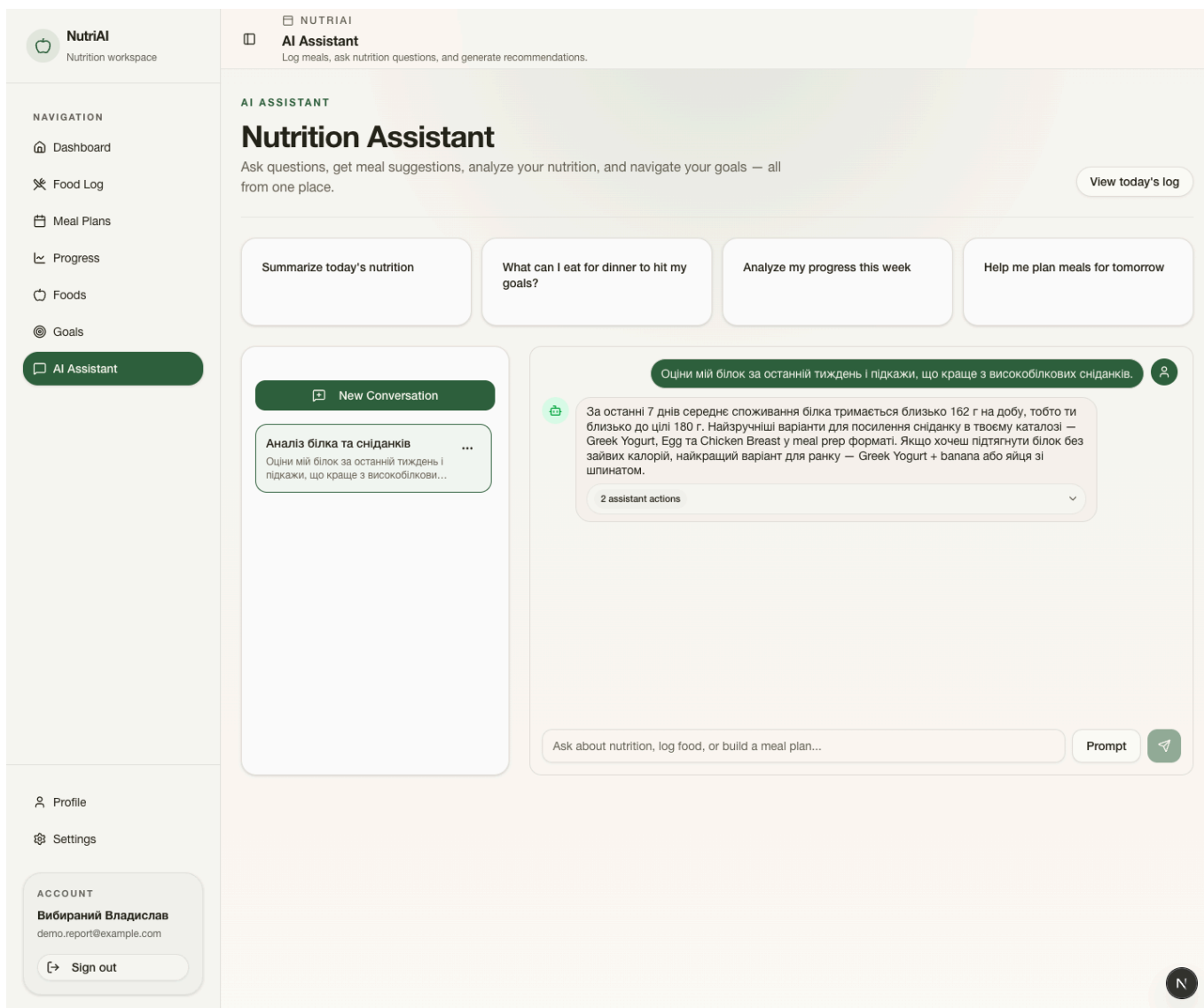


Рисунок 10: AI-асистент із збереженою розмовою та відображенням інструментальної активності.

5.3. Сценарії використання AI-асистента

AI-асистент у межах NutriAI виконує роль доменного помічника, а не універсального текстового співрозмовника. Типові сценарії використання:

1. пошук високобілкових продуктів у власному каталозі;
2. оцінка поточного відхилення від калорійної або білкової цілі;
3. логування їжі через natural language;
4. пояснення тижневого прогресу;
5. підказки щодо корекції meal plan.

Практична цінність такого підходу особливо помітна там, де користувачеві складно швидко інтерпретувати числові показники. Наприклад, замість самостійного пошуку причини недобору білка користувач може поставити запит у природній мові й отримати відповідь, побудовану на фактичних даних свого акаунта.

5.4. Переваги та обмеження реалізованої системи

До основних **переваг** реалізованого рішення належать:

1. наскрізна доменна модель без розриву між meal planning, food log і analytics;
2. типізований full-stack підхід, що зменшує кількість неузгодженостей між фронтендом і бекендом;
3. достатньо гнучкий AI-модуль із контрольованим набором інструментів;
4. відтворюваний сценарій формування демо-даних і скріншотів для документації.

Водночас система має і **обмеження**:

1. каталог продуктів поки не інтегровано з великими зовнішніми базами;
2. рекомендаційна частина AI спирається переважно на rule-informed interpretation, а не на окрему оптимізаційну модель;
3. tool-calling підхід не вирішує повністю проблему недетермінованості LLM;
4. критичні для production сценарії безпеки потребують глибшого аудиту й додаткових guardrails.

5.5. Технічні ризики

Для подальшого розвитку системи важливо враховувати такі технічні ризики:

1. latency і вартість запитів до зовнішніх AI-моделей;
2. помилки інтерпретації природної мови;
3. ризик помилкового логування їжі, якщо запит користувача двозначний;
4. вимоги до точнішої валідації інструментальних дій;
5. потребу у відстеженні джерела рекомендацій асистента.

5.6. Каркас для порівняння різних методів побудови агентів

Нижче наведено структурну заготовку для подальшого експериментального порівняння підходів до створення AI-агентів у межах nutrition assistant. На цьому етапі таблиця є каркасом і не містить вигаданих числових результатів.

Підхід	Реалізація	Керованість	Затримка	Вартість	Точність дій	Придатність
Single-agent	—	—	—	—	—	—
Tool-calling agent	—	—	—	—	—	—
Workflow agent	—	—	—	—	—	—
Multi-agent	—	—	—	—	—	—
OpenAI Agents SDK	—	—	—	—	—	—
LangGraph / LangChain	—	—	—	—	—	—
AutoGen	—	—	—	—	—	—
Власна реалізація NutriAI	—	—	—	—	—	—

Таблиця 3: Каркас порівняння підходів до побудови AI-агентів.

Під час майбутнього розширення цього розділу доцільно:

1. зафіксувати однакові користувацькі сценарії для всіх підходів;
2. вимірювати середній час відповіді та кількість інструментальних кроків;
3. враховувати не лише якість тексту, а і точність доменних дій;
4. порівнювати передбачуваність, дебагованість і зручність інтеграції у веб-застосунки.

5.7. Методика майбутнього експериментального порівняння

У майбутньому для кількісного дослідження можна використати такий план:

1. сформувати набір типових nutrition tasks: пошук продукту, аналіз тижневого прогресу, логування прийому їжі, рекомендація щодо білка;
2. запустити однакові завдання на різних агентних підходах;
3. заміряти latency, кількість tool calls, success rate, кількість некоректних дій;
4. провести експертну оцінку якості пояснення результатів;
5. проаналізувати engineering overhead кожного підходу.

5.8. Висновки до розділу

Реалізований проєкт демонструє, що модель планування харчування та моніторингу калорій може бути успішно інтегрована з AI-асистентом у межах сучасного full-stack веб-застосунку. Практична частина показує не лише наявність окремих функцій, а й узгоджену архітектуру, де аналітика, доменна модель та AI працюють як єдина система.

6. ВИСНОВКИ

У кваліфікаційній роботі розроблено модель планування харчування та моніторингу споживання калорій для AI-асистента і реалізовано її у вигляді веб-застосунку NutriAI. У процесі роботи проаналізовано предметну область цифрового self-tracking у сфері харчування, описано сутності користувача, продуктового каталогу, журналу споживання, планів харчування, цілей та AI-взаємодії.

У теоретичній частині обґрунтовано вибір tool-calling підходу для побудови прикладного AI-асистента, розглянуто роль контексту, пам'яті, guardrails та безпеки при інтеграції LLM у веб-застосунок. Показано, що найбільш доцільним для NutriAI є саме контрольований інструментальний доступ до доменних даних.

У практичній частині спроектовано і реалізовано full-stack архітектуру на основі Next.js, Bun, Prisma, PostgreSQL, Better Auth, Elysia та React Query. Забезпечено модулі food log, meal planning, goals, profile, analytics і AI-chat, а також підготовлено відтворюваний сценарій генерації демонстраційних даних і скріншотів.

Практична цінність результатів полягає в тому, що створений застосунок може бути основою для подальшого розвитку nutrition assistant з розширеним харчовим каталогом, рекомендаційною логікою та порівнянням різних агентних архітектур. У роботі також підготовлено структурний каркас для майбутнього кількісного порівняння методів побудови AI-агентів.

7. СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

- [1] World Health Organization, «Healthy diet». Дата звернення: 18, Квітень 2026. [Online]. Доступний у: <https://www.who.int/news-room/fact-sheets/detail/healthy-diet>
- [2] World Health Organization, «Obesity and overweight». Дата звернення: 18, Квітень 2026. [Online]. Доступний у: <https://www.who.int/news-room/fact-sheets/detail/obesity-and-overweight>
- [3] Anthropic, «Introducing the Model Context Protocol». Дата звернення: 18, Квітень 2026. [Online]. Доступний у: <https://www.anthropic.com/news/model-context-protocol>
- [4] LangChain, «LangGraph». Дата звернення: 18, Квітень 2026. [Online]. Доступний у: <https://www.langchain.com/langgraph>
- [5] Microsoft, «AutoGen». Дата звернення: 18, Квітень 2026. [Online]. Доступний у: <https://github.com/microsoft/autogen>
- [6] OpenAI, «OpenAI Agents SDK». Дата звернення: 18, Квітень 2026. [Online]. Доступний у: <https://openai.github.io/openai-agents-python/>
- [7] Chip Huyen, *AI Engineering*. O'Reilly Media, 2025.
- [8] Vercel, «Next.js Documentation». Дата звернення: 18, Квітень 2026. [Online]. Доступний у: <https://nextjs.org/docs>
- [9] Oven, «Bun Documentation». Дата звернення: 18, Квітень 2026. [Online]. Доступний у: <https://bun.sh/docs>
- [10] Prisma, «Prisma Documentation». Дата звернення: 18, Квітень 2026. [Online]. Доступний у: <https://www.prisma.io/docs>
- [11] Better Auth, «Better Auth Documentation». Дата звернення: 18, Квітень 2026. [Online]. Доступний у: <https://www.better-auth.com/docs>
- [12] ElysiaJS, «Elysia Documentation». Дата звернення: 18, Квітень 2026. [Online]. Доступний у: <https://elysiajs.com/>
- [13] TanStack, «TanStack Query Documentation». Дата звернення: 18, Квітень 2026. [Online]. Доступний у: <https://tanstack.com/query/latest>
- [14] Mark Richards i Neal Ford, *Fundamentals of Software Architecture*. O'Reilly Media, 2020.

8. ДОДАТОК А. ОПИС РЕПОЗИТОРІЮ ТА КОМАНД ВІДТВОРЕННЯ

Основні команди для відтворення матеріалів пояснювальної записки:

1. `bun run report:seed` — формує демо-дані для бакалаврського звіту;
2. `bun run report:screenshots` — запускає локальний застосунок, виконує вхід під демо-користувачем і створює скріншоти в `bachelor-report/assets/screenshots/`;
3. `bun run report:build` — компілює Typst-документ у `bachelor-report/report.pdf`;
4. `bun run lint` і `bun run build` — мінімальні перевірки якості коду проєкту.

9. ДОДАТОК Б. ПРИМІТКИ ЩОДО ПОДАЛЬШОГО РОЗВИТКУ

1. розширити блок порівняння агентних підходів фактичними експериментами;
2. додати автоматичну валідацію відповідей AI на контрольному наборі nutrition tasks;
3. інтегрувати зовнішні каталоги продуктів або barcode lookup;
4. посилити безпеку AI-дій за допомогою окремих шарів policy validation.